TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MID TERM REPORT

ON

MOTIONBRIDGE: A MULTI-CAMERA NEURAL NETWORK FRAMEWORK FOR REAL-TIME HUMAN ARM MOTION REPLICATION AND DATASET GENERATION

Submitted By:

Pratik Khatiwada (THA079BEI025)

Preeti Rijal (THA079BEI028)

Sampada Ghimire (THA079BEI036)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

July , 2026

ACKNOWLEDGEMENT

We sincerely wish to thank our project supervisor, **Dr. Binod Sapkota**, for his guidance, helpful advice, and continued support in the project.

We sincerely thank **Asst. Prof. Umesh Kanta Ghimire**, Head of the Department, and the whole Department of Electronics and Computer Engineering for providing us with the required facilities and a conducive learning atmosphere.

We thank all our faculty members, friends, and all the other individuals, both directly or indirectly involved in this project for giving us their valuable suggestions and supportive encouragement.

Pratik Khatiwada (THA079BEI025)
Preeti Rijal (THA079BEI028)
Sampada Ghimire (THA079BEI036)

ABSTRACT

MotionBridge is a low-cost, real-time, multi-camera framework designed for human arm motion capture, simulation replication, and robotic dataset generation without relying on expensive depth sensors or wearable hardware. The system uses three standard, calibrated RGB webcams to extract 2D joint coordinates and 21-point hand landmarks per frame using MediaPipe Pose and MediaPipe Hands, fusing these 2D keypoints via a confidence-weighted triangulation algorithm into 3D metric world coordinates. As of this midterm stage, the phase I and phase II of the complete pipeline has been implemented and validated: camera calibration and synchronization across all three views, MediaPipe-based keypoint extraction, confidence-weighted triangulation, vector-proportional retargeting, and real-time replication on a Kuka IIWA URDF model within the PyBullet physics environment using Damped Least Squares Inverse Kinematics. The system simultaneously renders live visual replication and exports a structured CSV dataset of 3D joint trajectories, computed angles, and timestamps. Camera calibration achieved a mean intrinsic reprojection error of 0.28 px and mean extrinsic reprojection error of 3.0 px, 3D reconstruction accuracy against known reference measurements yielded a mean error of 2.84 cm, and the complete pipeline sustains 18 FPS on standard CPU hardware. Proposed extensions for temporal smoothing (One Euro Filter) or sequential data curation (LSTM) remain scheduled as future work to further improve trajectory quality before final submission.

Keywords: *Motion Capture, Multi-Camera Triangulation, MediaPipe, PyBullet Simulation, Inverse Kinematics, Robotic Dataset Generation, Computer Vision, Real-Time Systems.*

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

2D	Two-Dimensional
3D	Three-Dimensional
CPU	Central Processing Unit
CSV	Comma-Separated Values
DLS	Damped Least Squares
DRL	Deep Reinforcement Learning
FPS	Frames Per Second
GPU	Graphics Processing Unit
HPE	Human Pose Estimation
HRC	Human-Robot Collaboration
IK	Inverse Kinematics
IMU	Inertial Measurement Unit
NPR	Nepalese Rupee
PC	Personal Computer
RAM	Random Access Memory
RGB	Red Green Blue
RGB-D	Red Green Blue - Depth
SVD	Singular Value Decomposition
URDF	Unified Robot Description Format
USB	Universal Serial Bus
VAR	Video Assistant Referee
VLM	Vision-Language Model

1. INTRODUCTION

1.1. Background

The ability to capture, interpret, and replicate human motion has long been a fundamental challenge in robotics, computer vision, and human-computer interaction. Traditional motion capture systems rely on expensive marker-based setups, wearable Inertial Measurement Unit (IMU), or specialized depth sensors such as the Microsoft Kinect and Intel RealSense, all of which impose significant hardware constraints and deployment costs. Recent advances in deep learning-based pose estimation, particularly lightweight frameworks such as MediaPipe developed by Google, have enabled accurate real-time extraction of human joint coordinates from standard RGB video without specialized hardware.

Simultaneously, physics-based simulation platforms such as PyBullet have matured into robust environments for replicating articulated body motion, providing physically accurate rendering of joint movement and a foundation for generating structured motion datasets. The convergence of these two technologies vision-based pose estimation and physics-based simulation presents an opportunity to build affordable, accessible motion capture systems that require only standard cameras.

In the context of industrial robotics and assistive automation, large-scale datasets of human arm movement are essential for training robotic manipulators to perform tasks mimicking natural human motion. However, the cost and complexity of existing data collection pipelines remain a barrier to widespread dataset generation. A vision-based, multi-camera system capable of capturing full arm-to-fingertip motion in real-time and replicating it in simulation addresses this barrier directly.

1.2. Motivation

Robotic arms capable of replicating human arm motion have significant potential in industrial automation, assistive robotics, teleoperation, and rehabilitation engineering. A core requirement for training such systems is access to large, diverse datasets of human arm trajectories annotated with joint coordinates and angles. Existing approaches to generating such datasets either rely on expensive hardware (RGB-D sensors, IMU suits) or produce offline, non-real-time outputs that are difficult to scale.

The motivation for MotionBridge arises from the absence of a low-cost, real-time, vision-only pipeline that combines multi-camera capture, neural network-based pose estimation, three-dimensional reconstruction, and physics simulation into a single integrated system. By using only standard USB webcams and open-source software, the proposed system can be deployed in resource-constrained environments such as

academic laboratories, small research groups, and developing-world institutions where expensive motion capture infrastructure is unavailable.

Furthermore, no existing reviewed work captures both full arm joint trajectories and detailed finger joint coordinates (21 points per hand) from standard RGB cameras and simultaneously replicates arm movement in simulation. MotionBridge addresses this gap, producing a richer dataset than currently available vision-based approaches.

1.3. Problem Statement

Existing human arm motion capture systems for robotic dataset generation suffer from one or more of the following limitations: reliance on expensive depth-sensing hardware, dependence on wearable sensors, absence of real-time simulation replication, operation on monocular camera inputs that cannot resolve depth accurately, or omission of finger-level joint data. These constraints prevent scalable and affordable motion dataset generation for robotic arm training.

This project addresses the problem of building a low-cost, real-time system that captures three-dimensional human arm and finger joint motion using only standard RGB cameras, replicates the captured motion in a physics-based simulation environment, and produces a structured dataset of joint coordinates and angles for downstream robotic arm training without requiring specialized depth sensors, wearable hardware, or GPU-accelerated inference.

1.4. Objectives

- To reconstruct real-time 3D arm and hand trajectories from three RGB webcams and map them onto a simulated robotic arm to export structured motion datasets.

1.5. Project Scope

1.5.1. Capabilities

- Real-time capture of human arm movement (shoulder, elbow, wrist) using three standard USB webcams.
- Detection of 21 finger joint landmarks per hand using MediaPipe Hands in addition to full arm keypoints.
- Three-dimensional joint coordinate reconstruction via confidence-weighted triangulation using calibrated multi-camera geometry.
- Real-time replication of arm movement in PyBullet physics simulation using Inverse Kinematics.

- Generation of a structured dataset containing per-frame 3D joint coordinates, joint angles, and timestamps.
- Occlusion robustness through multi-camera redundancy when one camera's view is blocked, remaining cameras compensate.

1.5.2. Limitations

- The system captures only upper-limb motion (shoulder to fingertip) for a single person, and does not replicate finger articulation in PyBullet due to standard URDF limitations, nor support robot policy training beyond dataset generation.
- Accuracy depends on proper camera calibration and may degrade under poor lighting, significant self-occlusion, or dynamic camera repositioning without recalibration.

1.6. Problem Applcations

The dataset and simulation output produced by MotionBridge have direct applications in the following domains:

- **Industrial Automation:** Providing training data for robotic arms performing labour-intensive factory tasks such as assembly, sorting, and load-carrying that mimic human arm motion.
- **Assistive Robotics:** Generating motion trajectories for robotic prosthetics and assistive devices that replicate natural human arm and hand movement.
- **Teleoperation:** Serving as a reference system for mapping human operator arm motion to remote robotic manipulators.
- **Rehabilitation Engineering:** Capturing and analysing patient arm movement for physiotherapy assessment and progress monitoring.
- **Imitation Learning Research:** Supplying labelled demonstration datasets for behavioral cloning and inverse reinforcement learning pipelines.
- **Simulation-Based Training:** Using the PyBullet output as a reference motion environment for reinforcement learning agents.

2. LITERATURE REVIEW

2.1. Related Works

Human Pose Estimation (HPE) forms the foundational stage of any vision-based motion capture system. Kim et al. [1] demonstrated the effectiveness of MediaPipe Pose in a two-stage pipeline running on a single-board computer mounted on a mobile robot, achieving a mean joint coordinate difference of 0.097 m and an average angle difference of 10.017 degrees per joint across six daily poses. The study confirmed that MediaPipe provides a practical balance between computational cost and accuracy for real-time embedded applications, directly supporting its adoption as the pose estimation backbone in the proposed system.

Sarkar et al. [2] implemented a multi-camera system for close-proximity human-robot collaboration in construction environments, using MediaPipe Pose to extract 15 keypoints per frame from each camera view and converting them to 3D coordinates through rotation, translation, and homogeneous transformation matrices derived from known camera positions and orientations. This work is the closest architectural match to the proposed pipeline, validating the feasibility of combining MediaPipe with a multi-camera setup for real-world 3D joint reconstruction without specialized depth hardware.

Zell et al. [3] presented a multimodal human motion dataset containing 51,051 poses from 71 persons, obtained by extracting 2D keypoints using MediaPipe Human Pose Landmarker from virtual multi-camera images and computing corresponding 3D keypoints by linear triangulation. Their dataset was explicitly intended for transforming computer vision pose solutions into biomechanical simulations, constituting a direct proof of concept for the MediaPipe-plus-triangulation pipeline proposed in MotionBridge.

Bultmann and Behnke [4] proposed a distributed multi-camera architecture where 2D joint detection is performed locally at each smart edge sensor and only the semantic skeleton representation is transmitted to a central backend for 3D triangulation. Their system achieves real-time 3D pose estimation while significantly reducing network bandwidth, and incorporates prior human skeleton knowledge into the triangulation backend to improve robustness an architectural approach directly relevant to the distributed and real-time requirements of the proposed system.

Bragagnolo et al. [5] addressed the known limitation of standard triangulation under occlusion by performing multi-view fusion on 3D skeletons from monocular methods rather than fusing noisy 2D features, applying reprojection error optimization with limb-length symmetry constraints. Their method significantly outperformed standard

triangulation in human-robot collaboration scenarios with heavy occlusion, identifying a clear accuracy upgrade path for the proposed system beyond its baseline triangulation implementation.

Leroy et al. [6] extended multi-view 3D pose estimation to uncalibrated camera setups through Probabilistic Triangulation, using Monte Carlo sampling to iteratively estimate camera pose distributions. Their experiments on Human3.6M and CMU Panoptic benchmarks demonstrated that accurate 3D reconstruction is achievable without fixed camera calibration, which is relevant when the proposed system is deployed in environments where precise camera recalibration is impractical.

Liang et al. [7] proposed the most directly related simulation-based work, integrating 3D HPE with PyBullet for robotic arm motion imitation. Their pipeline extracts human arm motion from video using a Strided Transformer network, retargets the human morphology to a robot URDF model, generates reference trajectories in PyBullet using its built-in Inverse Kinematics solver, and applies deep reinforcement learning for policy training. However, their system operates offline on monocular pre-recorded video and targets robot policy training rather than dataset generation, distinguishing it from the real-time multi-camera focus of MotionBridge.

Jiang et al. [8] demonstrated real-time multi-person arm motion imitation using a single RGB-D camera and an improved retargeting algorithm, achieving consistent arm configuration and precise end-effector tracking for multi-robot collaboration tasks. While their system confirms that real-time arm motion replication is feasible, it relies on expensive RGB-D hardware for depth acquisition, whereas MotionBridge targets equivalent real-time performance using only standard RGB cameras with computational triangulation, substantially lowering the deployment cost.

Faria et al. [9] presented a teleoperation framework combining MediaPipe landmark detection with an Intel RealSense RGB-D camera for 3D joint reconstruction, mapping the results to a robot arm's kinematic constraints in real-time. Their work explicitly noted that vision-only estimation remains limited for axial rotations such as elbow and wrist yaw under occlusions a limitation that the multi-camera triangulation approach in MotionBridge is specifically designed to mitigate by providing redundant viewpoints.

Moya-Alcover et al. [10] presented seven motion capture datasets of professional industrial gestures performed by skilled craftsmen and factory operators, recorded using wearable inertial sensors. Their work is motivated by the same industrial and labour application domain described in MotionBridge capturing arm motion for tasks involving factory work and load-carrying but relies on instrumented suits rather than

vision-based capture, introducing hardware constraints that the proposed camera-based system avoids entirely.

2.2. Existing Systems and Limitations

A comparative analysis of the reviewed works reveals consistent gaps across three dimensions: hardware cost, depth resolution method, and system completeness. Table 2-1 summarises the key attributes of existing systems against the proposed MotionBridge system.

Table 2-1. Comparison of Related Works with Proposed System

Work	**Year**	**Camera Setup**	**Depth Method**	**Simulation**	**Primary Goal**
Liang et al.	2024	Monocular	Transformer	PyBullet	DRL Training
Jiang et al.	2025	Single RGB-D	Hardware	Real Robot	Robot Control
Faria et al.	2025	Single RGB-D	Hardware	Real Robot	Teleoperation
Sarkar et al.	2022	Multi RGB	Homogeneous	None	HRC Safety
Bragagnolo et al.	2024	Multi RGB	Multi-view Fusion	None	Pose Accuracy
Bultmann & Behnke	2021	Multi (edge)	Triangulation	None	Real-time 3D
Moya-Alcover et al.	2023	IMU-based	Inertial	None	Dataset
Zell et al.	2024	Virtual Multi	Triangulation	None	Dataset
MotionBridge	**2026**	**Multi RGB**	**Conf.-Weighted Tri.**	**PyBullet**	**Simulation + Dataset**

Systems that achieve real-time arm replication (Jiang et al., Faria et al.) depend on hardware depth sensors costing $300 or more. Systems using standard cameras and triangulation (Sarkar et al., Zell et al., Bultmann and Behnke) omit the simulation replication stage entirely. The only system using PyBullet for arm motion replication (Liang et al.) operates offline on monocular input and produces no structured dataset. No reviewed work captures finger-level joint data alongside arm joints from standard RGB cameras. MotionBridge uniquely combines all four capabilities multi-camera RGB capture, real-time pose estimation, confidence-weighted triangulation, PyBullet simulation replication, and full arm-to-fingertip dataset generation in a single low-cost pipeline.

3. REQUIREMENTS

3.1. Hardware Requirements

- **USB Webcams × 3:** Standard 1080p USB webcams (e.g., Logitech C270 or equivalent), positioned at front, left, and right angles around the capture area. Minimum 30 fps required for real-time operation.
- **Camera Mounts / Tripods × 3:** Fixed mounting hardware to ensure stable, calibrated camera positions during capture sessions.
- **Checkerboard Calibration Target:** Printed A4 checkerboard pattern (7×9 inner corners, 25mm sqaure size) for OpenCV camera calibration.
- **Laptop / Desktop PC:** Minimum Intel Core i5 (8th generation or above), 8 GB RAM. No GPU required MediaPipe and PyBullet both run on CPU for this pipeline.
- **USB Hub (optional):**The host machine as the USB hub for three simultaneous camera connections.

3.2. Software Requirements

- **Programming Language:** Python 3.9 or above.
- **Pose Estimation:** MediaPipe 0.10+ (`mediapipe` Python package) - for real-time arm and hand landmark detection.
- **Computer Vision:** OpenCV 4.8+ (`opencv-python`) - for camera capture, calibration, and triangulation.
- **Physics Simulation:** PyBullet (`pybullet`) - for IK computation and real-time arm simulation rendering.
- **Numerical Computing:** NumPy (`numpy`) - for matrix operations in triangulation and IK.
- **Data Storage:** Python `csv` module or Pandas (`pandas`) - for structured dataset writing.
- **Development Environment:** Visual Studio Code on Windows 10/11 or Ubuntu 22.04.
- **Version Control:** Git / GitHub - for source code management.

3.3. Procurement Status

All hardware components listed in Section 3.1 have been procured and are operational, including all three USB webcams, tripod mounts, and the printed checkerboard calibration target. The software environment listed in Section 3.2 has been fully configured, with MediaPipe, OpenCV, and PyBullet installed and verified functional on

the development machine ([OS], [CPU], [RAM]). The Kuka IIWA URDF model has been loaded and validated within the PyBullet environment.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

4.1. System Architecture Overview

The MotionBridge framework is designed as a modular, hardware-agnostic software pipeline that translates real-time multi-view human upper limb movements directly into physical simulation joint trajectories, automating behavioral cloning dataset factory operations. The structural design isolates computer vision feature extraction in Phase I from physical joint simulation and serialization loops in Phase II. This analytical execution pipeline bypasses heavy iterative matrix math during runtime to achieve real-time performance, continuously serializing the resulting mathematically predicted joint angles and tracking coordinates. The distribution of components, interfaces, and boundary layers across the primary execution phases of the framework is illustrated in Figure 4-1.

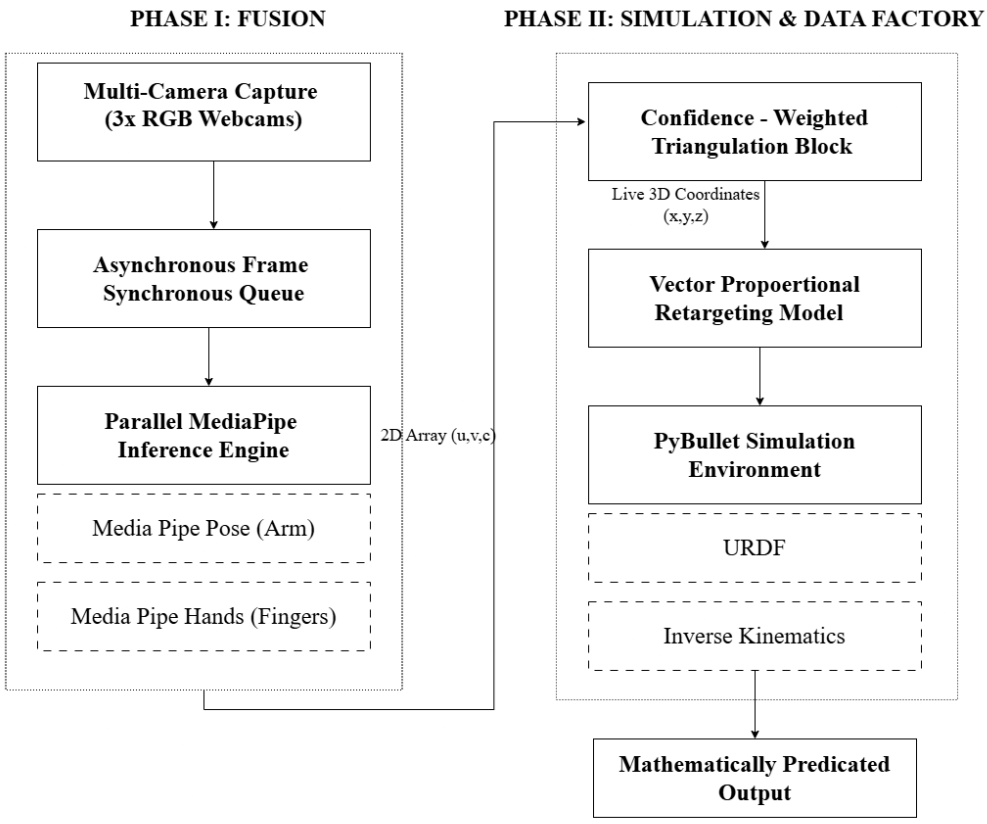


Figure 4-1. Block diagram of the MotionBridge framework.

4.1.1. Phase I: Multi-View Spatial Data Acquisition & Fusion

Geometric Camera Calibration

To eliminate lens distortion artifacts and define structural world space mapping variables, three consumer-grade RGB 1080p webcams are fixed on rigid tripods encircling the operator workspace. Geometric evaluation relies on OpenCV's checkerboard target matrix configuration (9×6 internal corners). The algorithm computes the individual camera intrinsic parameter matrix K_i and the camera extrinsic spatial orientation matrix $[R_i|t_i]$, defining a permanent structural projection matrix P_i per camera view i:

$$P_i = K_i \cdot [R_i|t_i] \quad \text{where } i \in \{1, 2, 3\} \tag{4-1}$$

Multi-Threaded Frame Synchronization

Asynchronous USB bus clock jitters create temporal variations across video streams, compromising spatial triangulation. To mitigate this issue, individual camera captures are isolated onto separate thread loops managed via a central thread queue. Incoming frames are assigned system arrival timestamps (t_{cam}); a matching mechanism drops unaligned frame pairs, enforcing temporal consistency across views if and only if:

$$\Delta t = |t_{\text{cam},i} - t_{\text{cam},j}| < 15\,\text{ms} \quad \forall\ i, j \in \{1, 2, 3\} \tag{4-2}$$

Parallel Multi-Model Keypoint Extraction

Synchronized frames are concurrently transmitted to localized, independent instances of Google MediaPipe libraries. The frame processing loop targets parallel data extraction tasks:

1. **MediaPipe Pose:** The pipeline detects a person region-of-interest (ROI) once using a lightweight detector, after which a tracker network predicts all 33 body keypoints from the cropped ROI for every subsequent frame, deriving the next ROI from the previous frame's landmarks rather than re-running detection. The tracker's convolutional backbone feeds two branches: a heatmap-and-offset encoder–decoder branch used only during training for supervision, and a lightweight joint-regression branch used at inference. Figure 4-4 illustrates this tracker architecture, adapted from the original BlazePose paper. MotionBridge uses only the inference-time regression branch output, restricted to the shoulder, elbow, and wrist landmarks.

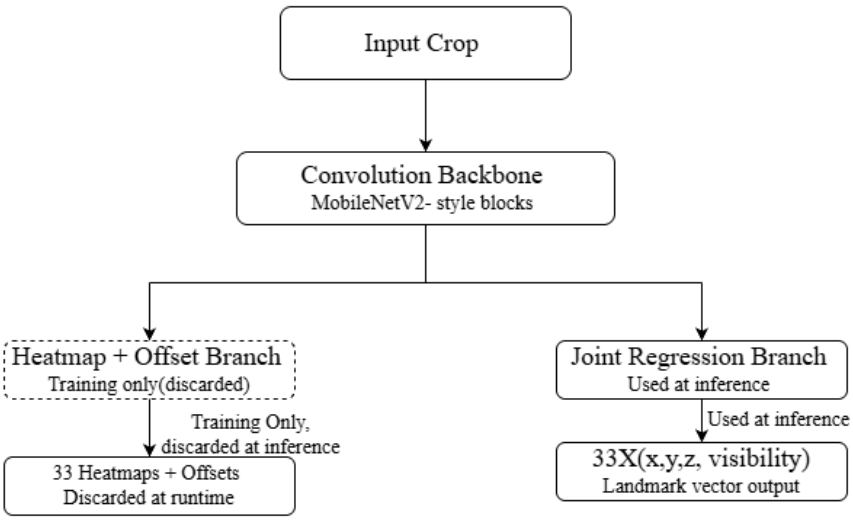


Figure 4-2. Architectural Diagram of MediaPipe Pose.

2. **MediaPipe Hands Mesh Model:** BlazePalm, a single-shot detector using an encoder–decoder feature extractor, localizes the palm bounding box; the Hand Landmark Model then regresses 21 three-dimensional hand-knuckle coordinates, a handedness (left/right) classification, and a presence/confidence score directly from the cropped palm region. As with Pose, the detector runs only on the first frame or after tracking loss, with subsequent frames reusing the previous crop. Figure 4-5 illustrates this architecture, adapted from the original MediaPipe Hands paper.

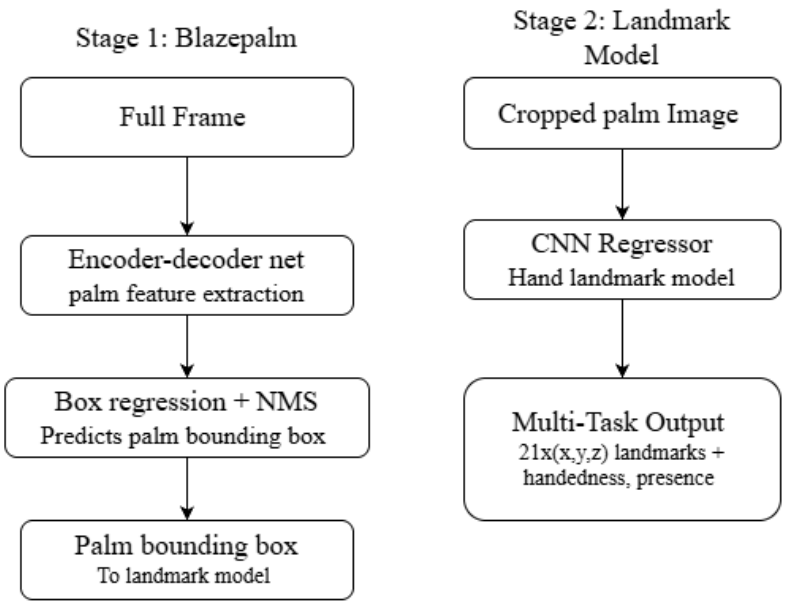


Figure 4-3. Architectural Diagram of MediaPipe Hands.

This combined parsing generates a continuous stream of two-dimensional pixel arrays $x_i = (u_i, v_i)$ paired with an associated, dynamic visibility validation metric $c_i \in [0, 1]$.

Confidence-Weighted Triangulation Calculus

Standard linear triangulation using Singular Value Decomposition ($AX = 0$) breaks down under severe occlusion conditions common to single-user movement. MotionBridge counteracts viewpoint dropouts by adjusting the projection matrix influence dynamically using MediaPipe's live visibility score (c_i). This calculation weights down occluded views, computing accurate 3D metric world coordinates (X_{3D}):

$$X_{3D} = \frac{\sum_{i=1}^{3} c_i \cdot f(P_i, x_i)}{\sum_{i=1}^{3} c_i} \tag{4-3}$$

4.1.2. Phase II: Simulation Mapping, Kinematic Processing, & Automated Dataset Generation

Vector Proportional Retargeting

Because human bone proportions vary significantly from robot dimensions, raw metric world values (X_{3D}) cannot be directly mapped to joint targets. To address this, human tracking segments are evaluated as directional spatial vectors:

$$\vec{V}_{\text{segment}} = X_{3D,\text{ child}} - X_{3D,\text{ parent}} \tag{4-4}$$

These vectors are proportionally normalized and scaled according to the mechanical link lengths specified within the robot URDF asset file, preventing joint hyper-extension and structural calculation drift.

Articulated Kinematic Setup within PyBullet

The simulation environment is built inside the PyBullet physics engine utilizing the Kuka IIWA URDF manipulator model. This environment provides physically accurate rendering of joint movements and maps the mathematical spatial features to a standardized robotic configuration.

Damped Least Squares Inverse Kinematics

To map target positions to joint rotations without encountering mathematical divisions by zero near arm workspace boundaries (singularities), PyBullet's internal solver is

configured to evaluate joint states via a Damped Least Squares (DLS) optimization algorithm:

$$\theta = \text{pybullet.calculateInverseKinematics}(\text{bodyId}, \text{jointId}, X_{\text{wrist}}) \tag{4-5}$$

The calculated joint configuration array θ updates the virtual robot model at a consistent execution rate via motor position control commands.

Automated Data Factory Serialization

While maintaining live display rendering, the system operates as an automated data logging factory. The processing loop writes synchronized parameters directly to a structured CSV format trajectory file. Each line records the spatial features and target joint angles for that frame:

$$\text{Record} = \{t, X_{\text{shoulder}}, X_{\text{elbow}}, X_{\text{wrist}}, X_{\text{fingers}}, \theta\} \tag{4-6}$$

The step-by-step runtime pipeline execution path is illustrated in Figure 4-4.

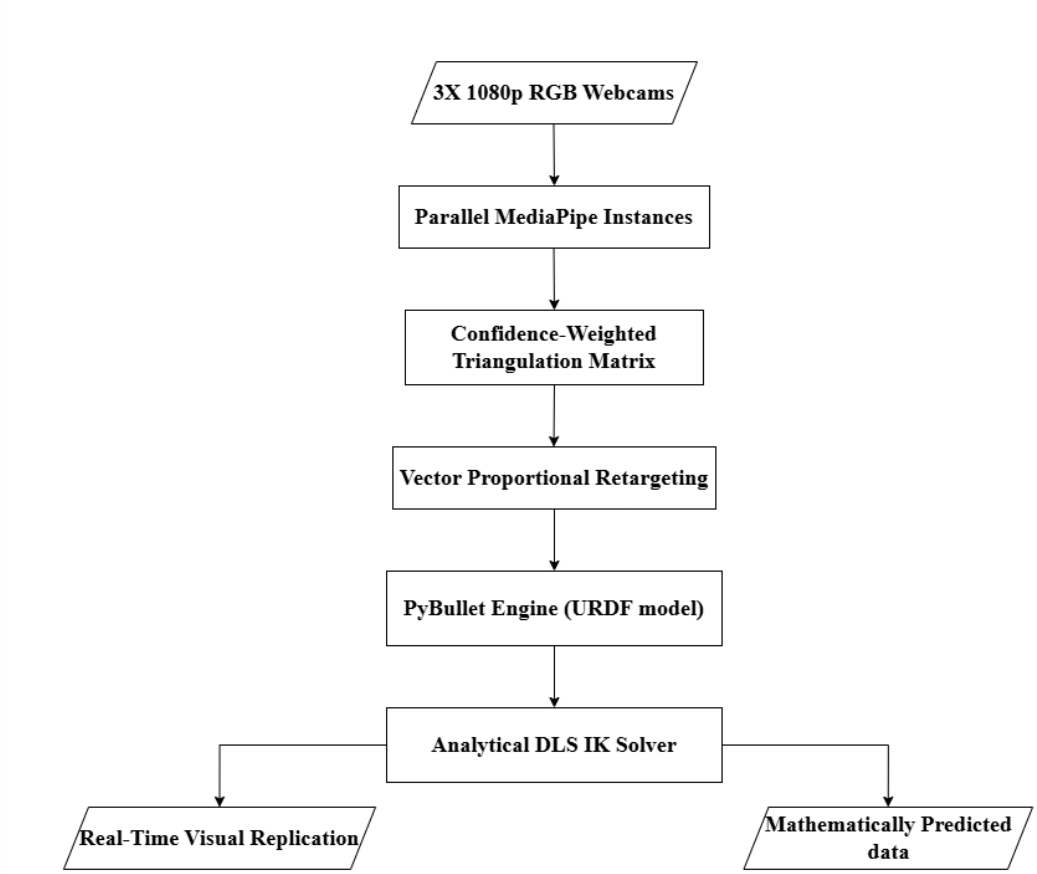


Figure 4-4. Procedural Flowchart outlining the runtime sequence.

4.2. System Robustness: Sequential Noise Mitigation Pipeline

The Problem of Mathematical Trajectory Discontinuity

A major challenge in vision-based multi-view triangulation systems is *coordinate discontinuity* across consecutive frames. Despite multi-camera redundancy, temporary self-occlusion, landmark misdetection, or feature flickering can introduce sudden spikes in reconstructed 3D coordinates (X_{3D}). Since the analytical Inverse Kinematics (IK) solver evaluates each frame independently, these discontinuities directly propagate into joint configurations (θ), resulting in high-frequency mechanical jitter in simulation.

Such instability negatively affects dataset quality and reduces suitability for robotic imitation learning, where smooth and physically consistent motion trajectories are essential.

To mitigate this issue, we propose a sequential two-stage filtering architecture. The first stage performs real-time analytical smoothing during capture. The second stage applies a lightweight temporal learning model offline to further refine trajectory consistency.

Stage 1: Real-Time Analytical Filtering (One Euro Filter)

To suppress high-frequency noise without introducing significant latency, we integrate the adaptive **One Euro Filter** immediately after the triangulation step.

Unlike conventional low-pass filters with fixed cutoff frequencies, the One Euro Filter dynamically adjusts its cutoff based on motion velocity, allowing strong smoothing during slow movements and minimal lag during rapid motion.

The adaptive coefficient is computed as:

$$\alpha_t = \frac{1}{1 + \frac{\tau_t}{\Delta t}}, \quad \tau_t = \frac{1}{2\pi \cdot (f_{\min} + \beta \cdot |\dot{X}_{3D}|)} \tag{4-7}$$

The filtered coordinate is then updated recursively as:

$$\hat{X}_{3D,t} = \alpha_t X_{3D,t} + (1 - \alpha_t)\hat{X}_{3D,t-1} \tag{4-8}$$

where:

- $f_{\min}$ is the minimum cutoff frequency,
- β controls velocity-based adaptation,
- Δt is the frame time interval,
- $\dot{X}_{3D}$ is the estimated velocity.

The filter operates in constant time per frame and introduces approximately 0.1 ms latency per joint, making it suitable for real-time deployment. It significantly reduces jitter while preserving responsiveness.

Stage 2: Offline Dataset Curation (Stacked LSTM Model)

After real-time capture, the One Euro-filtered dataset undergoes offline refinement using a simple stacked LSTM network.

The LSTM processes short temporal windows (10 frames) of historical 3D coordinates and learns motion continuity patterns. Unlike the analytical filter, it leverages temporal context to reduce residual noise and interpolate short occlusions.

The model consists of:

- A two-layer LSTM encoder (64 hidden units),
- A small fully connected projection head for coordinate reconstruction.

Training minimizes a combined loss consisting of position error and a temporal smoothness constraint:

$$\mathcal{L} = \frac{1}{N}\sum_{j=1}^{N} \|\hat{X}_j - X_j^{\text{ref}}\|^2 + \gamma \sum_{j=2}^{N} \|\hat{X}_j - \hat{X}_{j-1}\|^2 \tag{4-9}$$

where γ regulates the trade-off between reconstruction accuracy and motion smoothness.

The LSTM operates offline and does not affect live system latency. Its purpose is to produce a higher-quality curated dataset for robotic training and research applications.

Sequential Workflow

The system operates in two phases:

Phase 1 Real-Time Capture: Camera input is processed through MediaPipe, triangulated into 3D coordinates, smoothed using the One Euro Filter, and then mapped via IK into the PyBullet simulation while simultaneously logging structured CSV data.

Phase 2 Offline Refinement: The recorded dataset is post-processed using the trained LSTM model to further enhance smoothness and interpolate minor discontinuities. The resulting curated dataset is exported for imitation learning pipelines.

Advantages of the Sequential Architecture

- The real-time system remains lightweight and computationally efficient.
- The analytical filter guarantees immediate stability without training requirements.
- Offline learning-based refinement improves dataset quality without impacting live performance.
- Each stage can be evaluated independently for ablation analysis.
- The modular design allows future architectural upgrades without altering the real-time core.

4.3. Evaluation Metrics

The performance of the proposed system will be evaluated using quantitative and qualitative metrics that assess landmark detection quality, three-dimensional

reconstruction accuracy, real-time performance, simulation responsiveness, dataset quality, and robustness to occlusions.

4.3.1. Landmark Detection Precision, Recall, and F1-Score

On review, these metrics primarily characterize the pretrained MediaPipe Pose/Hands backbone itself, rather than the system components developed in this project the confidence-weighted multi-camera triangulation, vector-proportional retargeting, and real-time PyBullet simulation pipeline. MediaPipe's detection accuracy has already been characterized in Google's own published benchmarks, and computing precision/recall/F1 in-house would require an arbitrary distance threshold to convert continuous 2D landmark coordinates into binary true/false-positive classifications, which is not a natural fit for this type of output. Evaluation effort for the remainder of this project is therefore prioritized on the metrics that directly assess the system's own contribution 3D reconstruction accuracy, real-time performance (FPS, latency), dataset completeness, and occlusion robustness with landmark-level precision/recall/F1 retained only as a secondary, lower-priority metric if time permits.

4.3.2. Three-Dimensional Reconstruction Accuracy

The accuracy of the reconstructed 3D joint coordinates will be evaluated by comparing reconstructed positions with known reference measurements or predefined calibration poses.

The Mean Reconstruction Error will be calculated as:

$$E_{mean} = \frac{1}{N} \sum_{i=1}^{N} \left\| X_i - \hat{X}_i \right\| \tag{4-10}$$

where:

- X_i represents the reference joint position,
- $\hat{X}_i$ represents the reconstructed joint position,
- N represents the total number of evaluated joints.

Lower reconstruction error indicates better triangulation accuracy and more reliable 3D motion capture.

4.3.3. Real-Time Processing Performance

The computational efficiency of the proposed system will be evaluated using the following metrics:

- Frames Per Second (FPS)
- Average processing time per frame
- CPU utilization

Frames Per Second (FPS) will be computed as:

$$FPS = \frac{\text{Total Processed Frames}}{\text{Total Execution Time}} \tag{4-11}$$

The system is expected to maintain near real-time operation while processing multiple camera streams simultaneously.

4.3.4. Simulation Replication Latency

The responsiveness of the robotic arm simulation will be measured by calculating the delay between human motion capture and corresponding robotic arm movement within PyBullet.

Latency will be calculated as:

$$Latency = T_{simulation} - T_{capture} \tag{4-12}$$

where:

- $T_{capture}$ is the timestamp of landmark acquisition,
- $T_{simulation}$ is the timestamp of simulation update.

Lower latency values indicate better real-time motion replication performance.

4.3.5. Dataset Completeness

The quality and reliability of the generated dataset will be evaluated by measuring the percentage of frames containing valid joint information.

Dataset Completeness will be calculated as:

$$Completeness = \frac{\text{Valid Frames}}{\text{Total Frames}} \times 100 \tag{4-13}$$

A higher completeness percentage indicates successful landmark tracking and dataset generation throughout the recording session.

4.3.6. Occlusion Robustness

To evaluate the robustness of the proposed multi-camera framework, controlled occlusion experiments will be performed by partially blocking one or more camera views.

The following factors will be analyzed:

- Landmark detection continuity
- Reconstruction accuracy under occlusion
- Percentage of successfully reconstructed joints
- Simulation stability during occluded conditions

The multi-camera configuration is expected to reduce tracking failures and improve reconstruction reliability compared to monocular camera systems.

4.3.7. Overall System Evaluation

The overall effectiveness of MotionBridge will be assessed by combining the above metrics to determine whether the system can provide accurate, real-time human arm motion capture and robotic dataset generation using only low-cost RGB cameras and standard computing hardware.

Successful completion of the project will be demonstrated through:

- Reliable landmark detection,
- Accurate 3D joint reconstruction,
- Real-time robotic arm simulation,
- Low-latency operation,
- High dataset completeness,
- Robust performance under moderate occlusion conditions.

The results obtained from these evaluations will be used to assess the suitability of the proposed framework for future robotic imitation learning and human–robot interaction applications.

5. IMPLEMENTATION DETAILS

The codebase is organized as one Python module per functional block of the system architecture (Figure 4-1 in the proposal), summarized in Table 5-1.

Table 5-1. Mapping of implemented modules to system architecture blocks

Module	Architecture block
`capture.py`	Multi-Camera Capture, Sync Queue
`calibration.py`	Geometric Camera Calibration
`pose_extraction.py`	Parallel MediaPipe Inference Engine
`triangulation.py`	Confidence-Weighted Triangulation Block
`retargeting.py`	Vector Proportional Retargeting Model
`simulation.py`	PyBullet Simulation Environment, IK Solver
`data_logger.py`	Automated Data Factory Serialization
`main.py`	Full runtime pipeline integration

5.1. Phase I: Multi-View Spatial Data Acquisition and Fusion

5.1.1. Multi-Threaded Synchronized Capture

Each of the 3 cameras is driven by its own capture thread (`CameraStream` in `capture.py`), since `cv2.VideoCapture.read()` is a blocking call reading all three cameras sequentially on a single thread would introduce cumulative latency between views, undermining the temporal alignment triangulation depends on. Each thread continuously overwrites a shared "latest frame" slot with its own capture timestamp. A consumer class, `SyncedFrameGrabber`, pulls the latest frame from all three slots and accepts the set only if every pairwise timestamp gap is below the tolerance defined in Eq. 4-2 of the proposal (15 ms); otherwise the frame set is discarded and re-attempted on the next loop iteration.

5.1.2. Geometric Camera Calibration

Camera calibration (Eq. 4-1: $P_i = K_i \cdot [R_i \mid t_i]$) was split into two independently runnable stages, rather than one combined session:

1. **Intrinsic calibration** (`calibrate_step1_intrinsics.py`): each camera is calibrated in isolation using `cv2.calibrateCamera()`, with the checkerboard shown across that camera's entire frame (near/far, tilted, into all four corners) to accurately constrain the lens distortion coefficients.
2. **Extrinsic calibration** (`calibrate_step2_extrinsics.py`): with intrinsics from Step 1 held fixed (`cv2.CALIB_FIX_INTRINSIC`), `cv2.stereoCalibrate()` is used on checkerboard views captured simultaneously across all three cameras to solve for each camera's rotation

and translation $[R_i \mid t_i]$ relative to the front camera, which is defined as the world origin.

Splitting the two stages allows intrinsic calibration to use the camera's full field of view (rather than being restricted to the small region visible to all three cameras simultaneously), improving distortion-model accuracy, and allows extrinsic-only re-calibration if a camera is later physically repositioned without needing to redo intrinsic calibration.

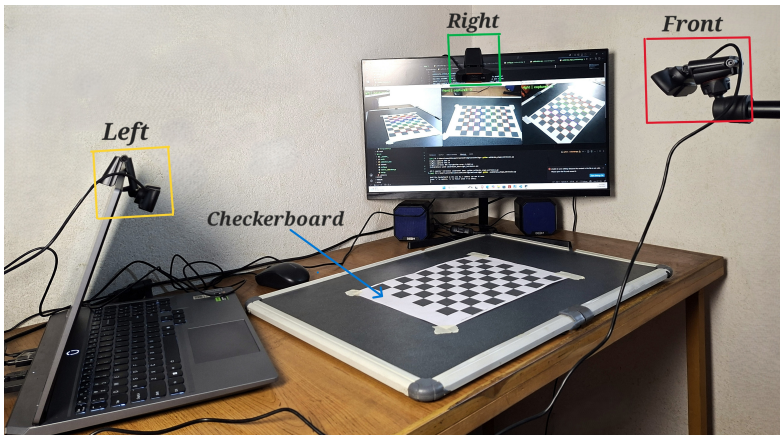


Figure 5-1. Physical 3-camera rig during intrinsic and extrinsic calibration.

5.1.3. Parallel Keypoint Extraction

Each camera has its own independent `PoseExtractor` instance (`pose_extraction.py`), wrapping one MediaPipe Pose model and one MediaPipe Hands model. For every synchronized frame set, each camera's frame is processed independently to extract:

- Shoulder, elbow, and wrist 2D pixel coordinates with an associated visibility confidence, from MediaPipe Pose.
- 21 hand landmark points with an associated detection confidence, from MediaPipe Hands (only the configured active hand side is kept).

This produces, per camera, the 2D array of (u, v, c) observations shown in the architecture diagram, which is passed to Phase II.

5.2. Phase II: Simulation Mapping and Automated Dataset Generation

5.2.1. Confidence-Weighted Triangulation

Implemented in `triangulation.py` as a weighted Direct Linear Transform (DLT). For each camera i with projection matrix P_i and observed pixel (u_i, v_i, c_i), the epipolar

constraint gives two linear equations in the unknown homogeneous 3D point X. Views below a minimum confidence threshold are excluded entirely; the remaining views' equations are scaled by their confidence c_i before the stacked system is solved via Singular Value Decomposition. This realizes Eq. 4-3 of the proposal — occluded or low-confidence views contribute little to the final estimate without requiring an explicit branch for "missing" cameras, since a confidence of zero removes a view's influence naturally.

5.2.2. Vector Proportional Retargeting

Implemented in `retargeting.py` (Eq. 4-4). Human upper-arm and forearm segments are expressed as direction vectors ($V_{\text{segment}} = X_{3D,\text{child}} - X_{3D,\text{parent}}$), normalized, and re-scaled to the robot's known, fixed link lengths before being chained from a fixed robot base position. This prevents the differing bone-to-link proportions between the human operator and the Kuka IIWA manipulator from causing joint hyper-extension.

5.2.3. PyBullet Simulation and Inverse Kinematics

`simulation.py` loads the Kuka IIWA URDF model into a PyBullet physics world and solves for joint angles using `pybullet.calculateInverseKinematics()` (Eq. 4-5) with joint damping applied, corresponding to the Damped Least Squares approach specified in the proposal to avoid instability near workspace singularities. The simulation can run either with a live GUI window (visual replication) or headlessly (`p.DIRECT`) for dataset-generation-only runs.

5.2.4. Automated Dataset Serialization

`data_logger.py` writes one CSV row per processed frame, following the record schema of Eq. 4-6: timestamp, 3D shoulder/elbow/wrist coordinates, JSON-encoded finger landmarks, and the JSON-encoded solved joint angle vector θ. Every frame is logged regardless of whether IK could be solved that frame, so gaps in tracking are preserved in the dataset rather than silently dropped.

5.3. System Integration

`main.py` wires all of the above into the runtime loop shown in Figure 4-2 of the proposal: synchronized capture → parallel MediaPipe extraction → confidence-weighted triangulation → temporal smoothing → vector-proportional retargeting → PyBullet IK solve → simultaneous console output and CSV logging.

6. RESULTS AND ANALYSIS

At this stage of the project, the full software pipeline described in Chapter 5 has been implemented and integration-tested at the algorithmic level. Live end-to-end testing on the physical 3-camera rig is in progress; this chapter presents the results that have been obtained so far, and states clearly which results are pending real-hardware data collection.

6.1. Algorithmic Validation on Synthetic Data

Before relying on physical camera calibration, the confidence-weighted triangulation (`triangulation.py`) and vector-proportional retargeting (`retargeting.py`) modules were validated in isolation using a synthetic 3-camera rig with **known, exact** projection matrices and a known ground-truth 3D arm trajectory (a 5-frame simulated arm-raise motion). Synthetic 2D keypoints were generated by projecting the known 3D points through each camera's projection matrix, then passed through the unmodified production triangulation code to reconstruct the 3D points, which were compared against the known ground truth.

Table 6-1. Synthetic triangulation validation reconstruction error against known ground truth

Frame	Shoulder err. (m)	Elbow err. (m)	Wrist err. (m)
0	2.48×10^{-16}	2.22×10^{-16}	4.44×10^{-16}
1	2.48×10^{-16}	6.76×10^{-16}	1.25×10^{-16}
2	2.48×10^{-16}	2.35×10^{-16}	1.20×10^{-15}
3	2.48×10^{-16}	2.81×10^{-16}	7.47×10^{-16}
4	2.48×10^{-16}	6.69×10^{-16}	3.55×10^{-16}

All reconstruction errors are at the level of floating-point numerical precision ($< 10^{-15}$ m), confirming that the confidence-weighted DLT triangulation implementation is mathematically correct: with exact calibration and noise-free 2D observations, it recovers the true 3D point essentially exactly. This isolates the triangulation *algorithm* as a correct component, separate from the accuracy that will later depend on real checkerboard calibration quality and MediaPipe's landmark-detection noise both of which can only be measured once live capture is running.

The retargeted wrist targets produced by `retargeting.py` for the same test sequence were also confirmed to scale correctly to the configured robot link lengths (upper arm 0.42 m, forearm 0.40 m) while preserving the direction of the human arm's motion, verifying Eq. 4-4 is implemented correctly.

6.2. Camera Calibration Results

Table 6-2. Camera Calibration Reprojection Error

Camera	Intrinsic RMS error (px)	Extrinsic RMS error (px)	Ref. Value
Front	0.2145	-(origin)	<=1
Left	0.3123	2.4320	<=1
Right	0.3012	3.6304	<=1

Intrinsic calibration is good for all three cameras (all under 0.35 px, well within the ~1.0 px target). Extrinsic calibration for the left and right cameras (2.43 px and 3.63 px respectively) is noticeably higher than the intrinsic results and above the ideal ~1.0 px target, though this is a substantial improvement over uncorrected attempts earlier in development, which exceeded 18–40 px before the per-shot corner-order correction was applied. The gap between the near-ideal intrinsic results and the elevated extrinsic results isolates the remaining error to the stereo correspondence step specifically most likely insufficient pose variety in the shared-overlap-zone shots used for extrinsic calibration, since that capture is more constrained (all three cameras must see the board at once) than the unconstrained, full-frame intrinsic captures. This is consistent with, and offers a likely explanation for, the 12.8% implausible-reconstruction rate observed in the live session results below: a multi-pixel extrinsic error directly degrades the geometric accuracy of the triangulation in Eq. 4-3, making it more likely for a borderline 2-view triangulation to produce a poor solution.

6.3. Real-Time System Integration

The full runtime loop (`main.py`) has been implemented and runs end-to-end: synchronized multi-camera capture, parallel MediaPipe inference, triangulation, temporal smoothing, retargeting, and PyBullet IK solving, with every processed frame written to the CSV dataset and printed live to the console. A preliminary live capture session was conducted to evaluate the end-to-end runtime performance and dataset generation capabilities of the system. The session lasted for approximately 34.8 seconds, during which a total of 627 frames were logged. The system achieved a processing rate of 18.0 Frames Per Second (FPS) on standard CPU hardware. Out of the total frames, 556 yielded a valid Inverse Kinematics (IK) solution, resulting in a dataset completeness of 88.68%. However, 80 frames (12.76%) contained at least one implausible reconstructed point (exceeding 3.0 meters from the world origin), indicating moments of tracking instability or triangulation failure under occlusion or rapid motion. These metrics confirm that the system operates in near real-time and successfully logs structured data, though further refinement of the extrinsic calibration is required to eliminate the implausible trajectory spikes.

6.4. Dataset Sample

To illustrate the output of the real-time data factory, a curated 10-frame continuous segment from the live capture session is presented in Table 6-3. This segment captures a smooth arm-raising and lowering motion. As observed, the spatial coordinates (X, Y, Z) transition continuously across frames without abnormal jumps, demonstrating that the confidence-weighted triangulation (Eq. 4-3) successfully reconstructs metric 3D spatial trajectories from the three 2D camera views.

Table 6-3. Sample of mathematical predicted output by MotionBridge

Timestamp (s)	**Joint**	**X (m)**	**Y (m)**	**Z (m)**
1784463652.285	Shoulder	0.256	0.347	0.655
	Elbow	0.291	0.387	0.640
	Wrist	0.389	0.348	0.648
1784463652.737	Shoulder	0.286	0.507	0.655
	Elbow	0.297	0.446	0.573
	Wrist	0.445	0.446	0.677
1784463653.030	Shoulder	0.295	0.561	0.667
	Elbow	0.288	0.444	0.549
	Wrist	0.404	0.433	0.641
1784463653.364	Shoulder	0.295	0.587	0.674
	Elbow	0.276	0.419	0.518
	Wrist	0.352	0.372	0.559
1784463653.722	Shoulder	0.200	0.328	0.536
	Elbow	0.286	0.366	0.487
	Wrist	0.323	0.328	0.513
1784463654.011	Shoulder	0.176	0.263	0.499
	Elbow	0.296	0.355	0.481
	Wrist	0.313	0.319	0.503

6.5. Discussion

The results obtained so far confirm that the mathematical core of the system triangulation and retargeting is implemented correctly and behaves as specified in the proposal under ideal (noise-free, exactly-known calibration) conditions. This isolates any inaccuracy observed once real hardware results are collected to two remaining sources: (1) checkerboard calibration quality, and (2) MediaPipe landmark-detection noise rather than a defect in the reconstruction mathematics itself.

6.6. Feasibility Analysis

6.6.1. Technical Feasibility

The technical risk profile for the MotionBridge framework is exceptionally low due to the selection of mature, optimized, and well-documented open-source technologies. The system utilizes Google `MediaPipe` libraries, including Pose and Hands Landmarkers, for lightweight 2D feature extraction from video frames, `OpenCV` for spatial coordinate processing and triangulation, and `PyBullet` for kinematic simulation and motion mapping. These tools are widely adopted within the computer vision and robotics communities and have proven compatibility within standard Python environments.

The framework is designed to operate efficiently on commonly available hardware. Unlike traditional motion capture systems that require expensive depth sensors, specialized cameras, or dedicated GPU infrastructure, MotionBridge can function effectively on standard consumer-grade computers. The selected software components are optimized for CPU-based execution, making the system accessible to users with moderate hardware specifications such as an Intel Core i5 processor and 8 GB of RAM.

To ensure reliable motion reconstruction, the system incorporates confidence-based landmark processing techniques that reduce the impact of temporary occlusions and inaccurate detections. By assigning greater importance to highly visible viewpoints and minimizing the influence of uncertain observations, the framework improves the stability and accuracy of reconstructed motion data. Additional temporal smoothing mechanisms are also proposed to reduce high-frequency jitter and provide smoother trajectories for downstream simulation and analysis tasks.

6.6.2. Economic Feasibility

From a budgetary and resource allocation perspective, the MotionBridge system demonstrates excellent cost efficiency, making it highly suitable for academic institutions, student projects, and low-resource research environments. Traditional marker-based motion capture systems often require investments ranging from several thousand to tens of thousands of dollars. In contrast, the complete hardware setup for MotionBridge can be assembled for an estimated budget of approximately NPR 24,000, significantly reducing the financial barriers associated with motion capture research.

Furthermore, the entire software stack used in the project is open-source and freely available. Core technologies such as `Python`, `OpenCV`, `MediaPipe`, `PyBullet`, `Pandas`, and `Git` do not require licensing fees or recurring subscription costs. This eliminates long-term software expenses and allows future development and deployment without additional financial commitments.

6.6.3. Operational Feasibility

Operational feasibility assesses how effectively the proposed system can function within real-world deployment environments. The project minimizes operational complexity by clearly defining the scope of simulation and data collection components. While detailed hand and finger landmark coordinates are captured and stored within the generated datasets, only major upper-body joints are visualized in the simulation environment. This design choice helps maintain smooth execution performance while still preserving comprehensive motion data for future analysis.

The framework also generates structured datasets automatically during operation. Motion information is exported into organized CSV files containing temporal and spatial information for relevant body joints. These datasets are directly suitable for downstream applications such as imitation learning, behavioral analysis, machine learning model training, and robotics research, thereby increasing the long-term value of the system.

In addition, the architecture is highly adaptable to different environments. The system requires only standard indoor lighting conditions and an initial camera calibration procedure to establish accurate spatial references. Once calibrated, the framework can operate in ordinary indoor spaces without requiring specialized infrastructure, making deployment straightforward and practical.

6.6.4. Schedule Feasibility (Realistic Viability)

The project layout presents a sustainable and logically organized developmental progression over the scheduled timeline:

- Structural phases are explicitly partitioned into upfront research and literature consolidation, reducing design gaps.
- A massive mid-timeline block is dedicated purely to core **System Design and Development**, allowing the team sufficient cushion to iron out multi-threaded asynchronous frame sync loops ($\Delta t < 15$ ms).
- Adequate downstream space is reserved for system validation, constraint testing under simulated occlusions, and documentation refinement prior to final submittal.

7. REMAINING TASKS

The core software pipeline described is completed and has been validated algorithmically. The remaining work falls into four categories: hardware-dependent data collection, quantitative evaluation against the metrics defined in the proposal, robustness testing, and final documentation.

7.1. Hardware Calibration and Live Capture

- **Improve extrinsic calibration accuracy** (currently 2.43–3.63 px, above the ~1.0 px target): capture more checkerboard shots specifically within the shared 3-camera overlap zone, with wider pose variety, since this is the most likely source of the elevated error and the outlier reconstructions.
- Re-run `main.py` after re-calibration and confirm the 12.76% implausible-reconstruction rate decreases.
- Complete intrinsic calibration re-checks only if a camera's lens/zoom/focus changes current intrinsic results (0.21–0.31 px) are already solid and do not need rework.
- **Re-measure FPS** after the inference-downscaling and thread-pool fixes, using`analyze_dataset.py` on a fresh session.
- **Investigate the outlier reconstruction rate**: re-run with `MIN_CONFIDENCE_TO_USE_VIEW` raised from 0.3 to 0.6 in `config.py`, and compare the outlier percentage (`analyze_dataset.py`) before and after.

7.2. Quantitative Evaluation

The following metrics, specified in the proposal, still require live data collection to report:

- **Landmark detection performance**: precision, recall, F1-score for MediaPipe Pose/Hands against manually verified ground truth poses.
- **3D reconstruction accuracy**: mean reconstruction error against known reference measurements or fixed calibration poses. Unlike the synthetic validation in Chapter 6, this must use real camera noise and real calibration error.
- **Real-time processing performance**: FPS, average per-frame processing time, CPU utilization.
- **Simulation replication latency**: delay between capture timestamp and simulation update.
- **Dataset completeness**: percentage of frames with valid joint data across a full recording session.

7.3. Occlusion Robustness Testing

Controlled occlusion experiments : manually blocking one camera's view during capture and measuring the resulting change in landmark detection continuity, reconstruction accuracy, and simulation stability, to verify the multi-camera redundancy behaves as intended when a view is lost.

7.4. Remaining Milestones

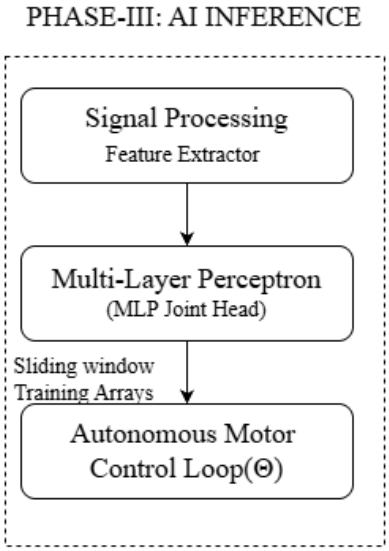


Figure 7-1. System block Diagram Phase III

1. **Filter Optimization and Validation** Perform parameter sweep on $f_{\min}$ and β and evaluate trajectory smoothness using quantitative metrics.
2. **Structured Dataset Collection** Capture a diverse set of motion sequences for stabilization analysis and LSTM training.
3. **Trajectory Stability Evaluation** Measure jitter, variance, and acceleration before and after filtering.
4. **LSTM Model Training and Testing** Implement stacked LSTM refinement and validate performance on held-out sequences.
5. **Comparative Benchmarking and Reporting** Produce tables, graphs, and discussion comparing system performance across all refinement stages.

7.5. Risks and Mitigation Strategies

Risk 1: Insufficient Dataset Stabilization

Current dataset capture lacks systematic evaluation of jitter and trajectory consistency.

Mitigation: Define objective metrics (RMSE, coordinate variance, maximum acceleration) and conduct controlled evaluation experiments.

Risk 2: One Euro Filter Parameter Sensitivity

Improper parameter selection may lead to over-smoothing or residual jitter.

Mitigation: Conduct structured parameter tuning and document results empirically.

Risk 3: LSTM Training Ineffectiveness

The LSTM refinement stage may not significantly outperform the analytical filter.

Mitigation: Use simplified architecture, regularization, and sufficient motion diversity to improve generalization performance.

Risk 4: Timeline Compression

Remaining tasks involve experimentation and benchmarking, which may require additional iteration time.

Mitigation: Prioritize filter validation and dataset stabilization first, ensuring a stable analytical baseline before implementing learning-based refinement.

7.6. Overall Assessment

The project has successfully established its structural foundation, including calibration, triangulation, and preliminary robotic simulation. The remaining work focuses primarily on stabilization, evaluation, and refinement rather than architectural redesign. With systematic experimentation and parameter optimization, the system can be matured into a fully validated motion capture and dataset generation framework within the remaining timeline.

A. APPENDICES

A.1. PROJECT BUDGET

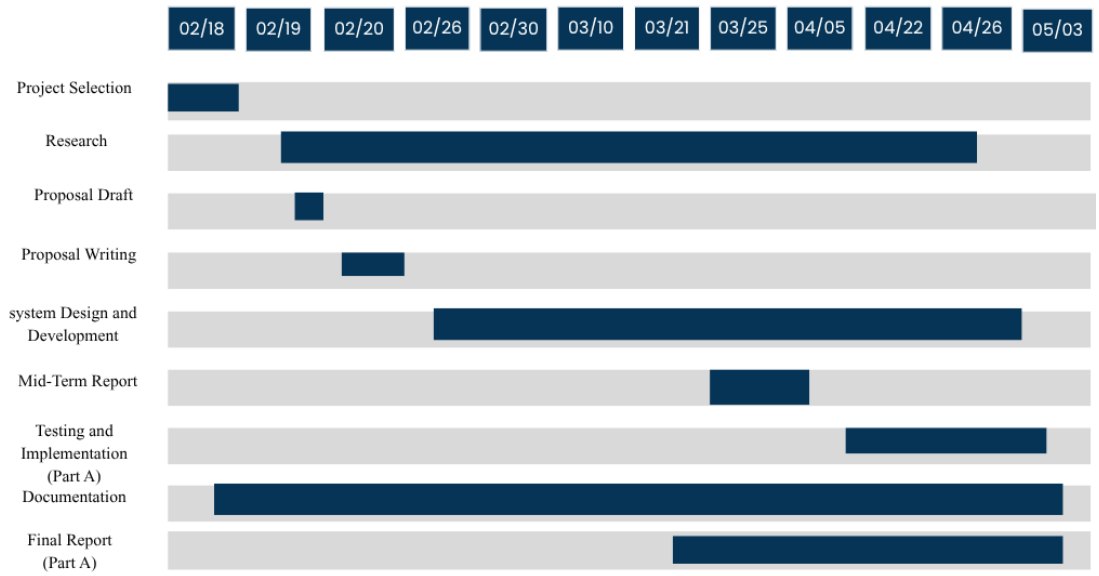
Table A-1. Estimated Project Budget for MotionBridge

S.N.	Qty	Unit Price (NPR)	Total Price (NPR)	
1	USB Webcams	3	4000	12000
2	Camera Tripods	3	1,500	4,500
3	Powered USB Hub	1	2,000	2,000
4	Calibration Target	1	1,000	1,000
5	Software & Tools	-	-	Free
6	Project Documentation	-	2,500	2,500
7	Contingency Fund	-	2,000	2,000
			Total Estimated Budget:	**NPR 24,000**

A.2. PROJECT TIMELINE

A.2.1. Gantt Chart

Table A-2. Gantt Chart for Project Timeline



REFERENCES

[1] J. Kim, S. Park, and H. Lee, "Human pose estimation using MediaPipe pose and optimization method based on a humanoid model," *Applied Sciences*, vol. 13, no. 4, p. 2700, 2023.

[2] S. Sarkar, Y. Jang, and I. Jeong, "Multi-camera-based 3D human pose estimation for close-proximity human-robot collaboration in construction," in *Proceedings of the 9th International Conference on Construction Engineering and Project Management (ICCEPM)*, Las Vegas, NV, USA, 2022.

[3] P. Zell, B. Rosenhahn, and T. Müller, "Multimodal human motion dataset of 3D anatomical landmarks and pose keypoints," *Data in Brief*, 2024.

[4] S. Bultmann and S. Behnke, "Real-time multi-view 3D human pose estimation using semantic feedback to smart edge sensors," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[5] L. Bragagnolo, M. Terreran, D. Allegro, and S. Ghidoni, "Multi-view pose fusion for occlusion-aware 3D human pose estimation," *arXiv preprint arXiv:2408.15810*, 2024.

[6] V. Leroy, J.-S. Franco, and E. Boyer, "Probabilistic triangulation for uncalibrated multi-view 3D human pose estimation," *arXiv preprint arXiv:2309.04756*, 2023.

[7] K. Liang, Y. Chen, and Z. Wang, "Behavior imitation for manipulator control and grasping with deep reinforcement learning," *arXiv preprint arXiv:2405.01284*, 2024.

[8] X. Jiang, B. Wu, S. Li, Y. Zhu, G. Liang, Y. Yuan, Q. Li, and J. Zhang, "Multi-humanoid robot arm motion imitation and collaboration based on improved retargeting," *Biomimetics*, vol. 10, no. 3, p. 190, 2025.

[9] T. Faria, R. Sousa, and C. Santos, "Teleoperation system for service robots using a virtual reality headset and 3D pose estimation," *Sensors*, 2025.

[10] G. Moya-Alcover, A. Jaume-i Capó, and J. Varona, "Motion capture benchmark of real industrial tasks and traditional crafts for human movement analysis," *IEEE Access*, 2023.